Laboratoire #0

Initiation à MATLAB & SIMULINK

Semaine 1

GBM3310 - Modélisation et commande en physiologie

Hiver 2026
Département de génie électrique
Polytechnique Montréal

Dernière mise à jour: 9 janvier 2026

Objectif Général

Le but de ce laboratoire est d'introduire l'étudiant·e au logiciel de calcul numérique MATLAB (Mat*rix* Lab*oratory*) ainsi que l'outil de simulation associé SIMULINK. Bien que les étudiant·e·s aient une certaine connaissance de MATLAB (mais probablement peu de SIMULINK), cette première séance de travaux pratiques se veut une mise à niveau afin que tous les étudiant·e·s aient les mêmes outils pour démarrer les laboratoires de GBM3310 - Modélisation et commande en physiologie. Les notions apprises lors de ce premier laboratoire seront constamment utilisées durant le semestre ; il est de la responsabilité de l'étudiant·e de maîtriser ces nouveaux outils.

1 Les bases de MATLAB et un peu plus...

Nous allons d'abord passer en revue quelques fonctions de MATLAB. Nous traiterons en premier des fonctions de base, ensuite des fonctions graphiques et enfin des fonctions propres à l'analyse des systèmes. Exécutez toutes les lignes de code qui vont suivre afin de vous familiariser avec leur utilisation. Pour avoir de l'aide en ligne sur une fonction, utilisez :

- la commande `help` suivi du nom de la fonction pour de l'aide succincte dans le terminal ;
- la commande `doc` suivi du nom de la fonction pour la documentation complète de la fonction dans une nouvelle fenêtre.

Si l'on souhaite que le résultat de l'instruction ne s'affiche pas, on termine la ligne de code par un point-virgule '`;`'. Vous avez le choix de taper vos lignes de commande directement dans le terminal (`Command Window`) ou dans un `Live Script` qui permet de visualiser et conserver vos instructions, ainsi que leurs résultats (Fig. 1)

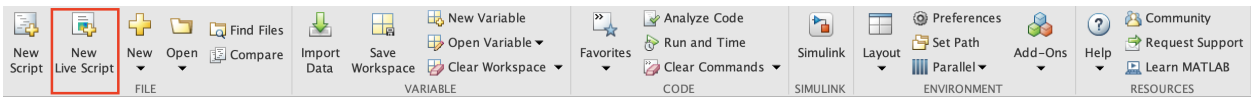


FIGURE 1 – Lancement d'un `Live Script`

1.1 Fonctions de base

On introduit ici les fonctions de base pour manipuler les variables scalaires et matricielles, et accéder aux éléments de ces matrices. Les matrices se rentrent ligne par ligne en séparant les éléments par un espace ou une virgule '`,`' ; on passe à la ligne suivante avec un point-virgule '`;`'.

1.1.1 Déclarations et sélections

Déclaration d'une matrice.

```
>> A = [1 2 3 4; 5 6 7 8; 9 10 11 12]
```

Déclaration d'une matrice identité (e.g., de dimension 4)

```
>> B = eye(4)
```

Déclaration d'une matrice remplie de zéros (e.g., de dimension 5)

```
>> C = zeros(5)
```

Déclaration d'une matrice remplie de uns (e.g., de dimensions 3×2)

```
>> D = ones(3,2)
```

Sélection d'un élément d'une matrice (e.g., l'élément $(3, 4)$, troisième ligne, quatrième colonne).

```
>> A(3,4)
```

Sélection d'une ligne complète d'une matrice (e.g., la deuxième). Remarquez l'utilisation de '`:`' pour signifier qu'on sélectionne toutes les colonnes.

```
>> A(2,:)
```

Sélection d'une colonne complète d'une matrice (e.g., la troisième). Remarquez l'utilisation de '`:`' pour signifier qu'on sélectionne toutes les lignes.

```
>> A(:,3)
```

Quels sont les résultats rendus par les instructions suivantes ?

```
>> A(2:3,3:4)
>> A([1 4],[1 2 4])
>> A([1 3],[1 2 4])
```

1.1.2 Modifications et Opérations

Transposition avec l'apostrophe ' '

```
>> B = A'
```

Modification d'un élément, d'une ligne ou d'une colonne d'une matrice. Les dimensions doivent être adéquates.

```
>> A(2,3) = -1
>> A(1,:) = [0 -1 -2 -3]
>> A(:,3) = [-4;-5;-6]
```

Que fait l'instruction suivante ?

```
>> A([1 3],[1 4]) = [10 10;10 10]
```

Les opérations arithmétiques peuvent bien évidemment être effectuées. En revanche dans le cas des matrices, il faut que les dimensions soient adéquates sinon MATLAB retourne un message d'erreur.

```
>> B = [1 2; 3 4]
>> C = [5 6 7]
>> B+C
>> B*C

>> D = [5 6; 7 8]
>> B+D
>> B*D
```

Dans le cas de la multiplication (et mise à la puissance) de matrices, on différenciera la mutliplication classique des matrices (telle que définie en algébre linéaire) et la multiplication élément à éléments pour laquelle on fait précéder le signe de multiplication '*' d'un point '.'.

```
>> B*B*B
>> B^3
>> B.*B.*B
>> B.^3
```

1.1.3 Fonctions

Une liste de fonctions élémentaires peut être trouvée en table 1.

TABLE 1 – Fonctions élémentaires

`abs`	Valeur absolue
`sqrt`	Racine carrée
`sign`	Fonction signe
`i,j`	$i^2 = j^2 = -1$
`pi`	Nombre π
`conj`	Conjugé d'un nombre complexe
`imag`	Partie imaginaire d'un nombre complexe
`real`	Partie réelle d'un nombre complexe
`angle`	Argument d'un nombre complexe
`cos`	Fonction cosinus
`sin`	Fonction sinus
`tan`	Fonction tangente
`exp`	Fonction exponentielle
`log`	Fonction logarithme naturel
`log10`	Fonction logarithme en base 10
`cosh`	Fonction cosinus hyperbolique
`sinh`	Fonction sinus hyperbolique
`tanh`	Fonction tangente hyperbolique
`acos`	Fonction cosinus inverse
`asin`	Fonction sinus inverse
`atan`	Fonction tangente inverse
`atan2`	Fonction tangente inverse à 4 quadrants
`acosh`	Fonction cosinus hyperbolique inverse
`asinh`	Fonction sinus hyperbolique inverse
`atanh`	Fonction tangente hyperbolique inverse
`det`	Déterminant d'une matrice
`eig`	Valeurs propres d'une matrice
`damp`	Valeurs propres d'une matrice
`trace`	Trace d'une matrice
`rank`	Rang d'une matrice
`inv`	Inverse d'une matrice
`size`	Dimensions d'une matrice
`length`	Taille d'un vecteur

1.2 Fonctions graphiques

Les graphiques sous MATLAB se font dans une fenêtre graphique créée par la commande `figure` ou quand on utilise une commande graphique spécifique (`plot`, `semilogx`, `semilogy`, `loglog`...). On peut créer autant de fenêtres que désirées ; elles sont numérotées de 1 à n au fur et à mesure de leur création. La fenêtre graphique active est la dernière qui a été créée par la commande `figure` ou qui a été activée par une commande ou qui a été sélectionnée par la souris.

```
>> figure
>> figure(4)
```

Création d'un vecteur d'éléments allant de 0 à 10 avec un échantillonnage de 0.1.

```
>> t = 0:0.1:10
```

Création des vecteurs de données correspondants.

```
>> y = 1+t.^2/5
>> z = 5*sin(t)
>> w = 20*exp(-t)
```

Tracé avec différents styles. Utilisez `help` pour connaître les différents styles et codes de couleur. La commande `hold` est nécessaire pour conserver les graphes au fur et à mesure sans écraser les précédents. La commande `grid` rajoute une grille sur le graphe.

```
>> plot(t,y,'-')
>> hold on
>> grid on
>> plot(t,z,'k--')
>> plot(t,w,'o')
```

Les commandes suivantes permettent de rajouter un titre, de nommer les axes, d'ajuster les dimensions des axes et de rajouter une légende. En ce qui concerne la légende, l'ordre des courbes respectent l'ordre dans lequel les graphes ont été tracés.

```
>> title('Premiers graphes')
>> xlabel('Temps (s)')
>> ylabel('Donnees')
>> axis([0,10,-5,25])
>> legend('y','z','w')
```

On obtient alors le résultat de la figure 2a.

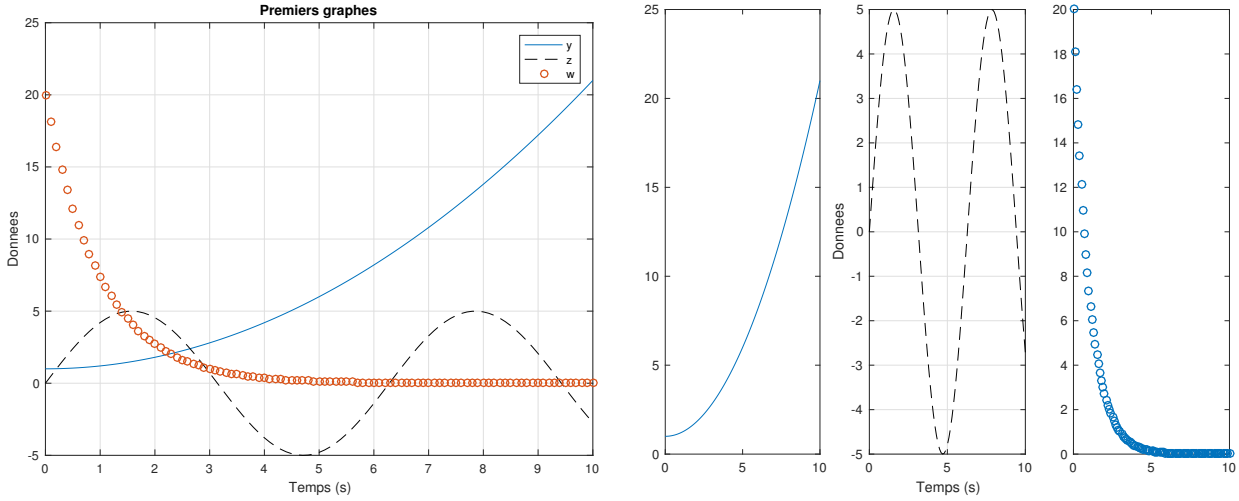
On peut tracer ces 3 courbes de façon séparée mais sur une seule et même figure. On a alors recours à la fonction `subplot(n,m,p)` qui subdivise la figure en $n \times m$ graphes (n lignes fois m colonnes). On se positionne sur le p-ième graphe pour faire le tracé. L'exemple qui suit donne le résultat de la figure 2b. Remarquez que les instructions `grid`, `xlabel` et `ylabel` n'ont effet que sur

le graphe actif. Il faudrait donc répéter ces instructions pour obtenir le même effet sur les autres graphes.

```
>> subplot(1,3,1)
>> plot(t,y,'-')
>> subplot(1,3,2)
>> plot(t,z,'k--')
>> grid on
>> xlabel('Temps (s)')
>> ylabel('Donnees')
>> subplot(1,3,3)
>> plot(t,w,'o')
```

On peut fermer la figure courante ou toutes les figures avec les instructions.

```
>> close
>>
3 close all
```



(a) Premiers graphes (b) Illustration de `subplot`

FIGURE 2 – Illustration des fonctions graphiques

2 Fonctions utiles pour l'analyse des systèmes

Nous serons amenés durant la session à étudier des fonctions de transfert et des modèles d'état dont nous souhaiterons obtenir certaines caractéristiques. Une fonction de transfert se présente sous la forme d'une fraction rationnelle de la variable s de Laplace et un modèle d'état linéaire sous la forme de 4 matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$. Il est alors possible d'en extraire les pôles, les zéros, les valeurs propres, etc.

2.1 Fonctions de transfert et modèles d'état

Il s'agit en premier lieu de rentrer des polynômes sous MATLAB. On stocke dans un vecteur les coefficients du polynôme par puissance descendante. La longueur du vecteur définit l'ordre du polynôme et il faut préciser tout coefficient nul. On définit par exemple le polynôme

$$p(s) = 6s^6 + 5s^5 + 3s^3 + 2s^2 + s$$

avec l'instruction suivante :

```
>> P=[6 5 0 3 2 1 0]
```

On obtient les racines de ce polynôme avec la fonction `roots` :

```
>> roots(P)
```

ou la fonction `damp`, qui a l'avantage de donner en plus la pulsation naturelle et l'amortissement des pôles complexes conjugés :

```
>> damp(P)
```

On désire stocker la fonction de transfert

$$F(s) = \frac{s^2 + s}{s^3 + 3s^2 + s + 1}$$

Il y a deux façons de définir une telle fonction de transfert en utilisant la fonction `tf`. La première façon consiste à déclarer le numérateur et le dénominateur sous la forme de vecteurs comme précédemment :

```
>> F=tf([1 1 0],[1 3 1 1])
```

La seconde façon consiste à déclarer la variable s de Laplace et de s'en servir pour construire la fonction de transfert :

```
>> s=tf('s')
>> F=(s^2+s)/(s^3+3*s^2+s+1)}
```

On peut alors obtenir les pôles, zéros et gain statique de la fonction de transfert et factoriser la fonction de transfert :

```
>> pole(F)
>> zero(F)
>> dcgain(F)
>> zpk(F)
```

On peut opérer différents calculs sur les fonctions de transfert (addition, multiplication, etc...). En revanche, il faut être vigilant car MATLAB ne procède pas aux simplifications. Comparez par exemple :

```
>> F=tf([1],[1 1])
>> G=F/(1+F)
```

et

```
>> F=tf([1],[1 1])
>> H=minreal(F/(1+F))
```

Vous voyez que `G` n'a pas été simplifiée, contrairement à `H`. Il faut donc penser à utiliser la fonction `minreal` pour forcer la simplification.

On considère enfin un modèle d'état linéaire :

$$\begin{cases} \dot{\mathbf{x}} = \mathbf{A}\mathbf{x} + \mathbf{B}\mathbf{u} \\ \mathbf{y} = \mathbf{C}\mathbf{x} + \mathbf{D}\mathbf{u} \end{cases}$$

L'utilisation de la fonction `ss` avec les matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$ permet de créer une variable renfermant le modèle d'état :

```
>> A=[0 1 0; 0 0 1; -1 -1 -3]
>> B=[0; 0; 1]
>> C=[0 1 1]
>> D=[0]
>> sys=ss(A,B,C,D)
```

Il est possible de retrouver les matrices $(\mathbf{A}, \mathbf{B}, \mathbf{C}, \mathbf{D})$ en utilisant les champs dédiés :

```
>> sys.a
>> sys.b
>> sys.c
>> sys.d
```

Si l'on veut les valeurs propres du système, on peut utiliser :

```
>> damp(sys)
>> eig(sys)
>> pole(sys)
```

On peut enfin passer du modèle d'état à la fonction de transfert (cas SISO) ou à la matrice de fonctions de transfert (cas MIMO) avec les fonctions `tf` ou `zpk` :

```
>> tf(sys)
>> zpk(sys)
```

2.2 Réponses temporelles

Étant donnés une fonction de transfert ou un modèle d'état, il est possible de visualiser la réponse du système à une impulsion (fonction `impulse`) ou un échelon unité (fonction `step`) :

```
>> F=1/(s^3+3*s^2+s+1)
>> impulse(F)
>> step(F)
```

Les réponses temporelles s'affichent dans des figures sur lesquelles il est possible de faire un clic droit pour obtenir les caractéristiques (`characteristics`) de la réponse temporelle :

- le dépassement et le temps de pic (`Peak Response`) ;
- le temps de réponse à 2% (`Settling Time`) ;
- le temps de montée de 10% à 90% de la valeur en régime permanent (`Rise Time`) ;
- la valeur en régime permanent (`Steady State`).

En cliquant sur la caractéristique voulue, un point bleu s'affiche sur la courbe et en passant le curseur dessus, les informations apparaissent. Dans le cas de la réponse à l'échelon unité, la fonction `stepinfo` permet d'obtenir directement ces renseignements :

```
>> stepinfo(F)
```

2.3 Représentations fréquentielles

Le diagramme de Bode fournit deux graphes, où l'on retrouve le gain (en dB) et la phase (en deg) de la fonction de transfert en fonction de la pulsation (en rad/s) :

```
>> F=1/(s^3+3*s^2+s+1)
>> bode(F)
```

Il est alors possible d'obtenir certaines caractéristiques avec un clic droit (`characteristics`). On retrouve le maximum de la réponse en gain avec `Peak Response`. Le point bleu qui apparaît fournit la valeur du maximum et la pulsation correspondante. Les autres options `Minimum Stability Margins` et `All Stability Margins` permettent d'obtenir les marges de gain, de phase et de retard. Ces marges seront abordés plus tard durant la conception des compensateurs dans le domaine fréquentiel.

3 SIMULINK

SIMULINK est l'outil qui va nous permettre de créer des modèles et de les simuler. Pour y accéder, tapez la commande `simulink` sous MATLAB ou cliquez sur l'icône de SIMULINK (Fig. 3)

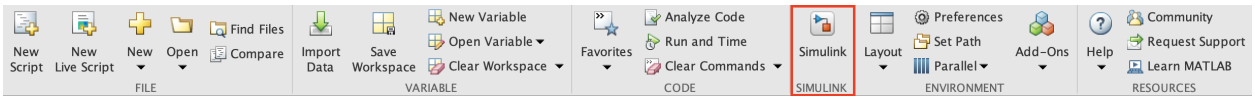


FIGURE 3 – Lancement de SIMULINK

La librairie de SIMULINK (*Library Browser*) contient les différentes sous-librairies (selon différents thèmes) contenant elles-mêmes les blocs qui vous serviront à bâtir vos diagrammes fonctionnels (Fig. 4).

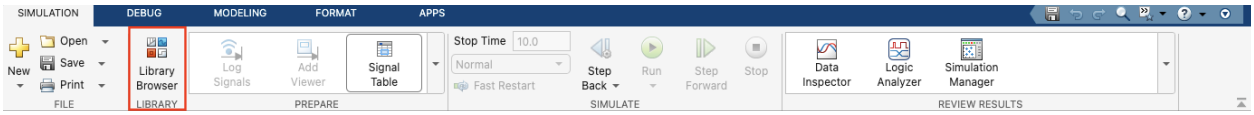


FIGURE 4 – Bibliothèque de SIMULINK

Il est possible de chercher directement un bloc en double-cliquant sur le fond blanc d'un schéma SIMULINK et en tapant les premières lettres (Fig. 5).

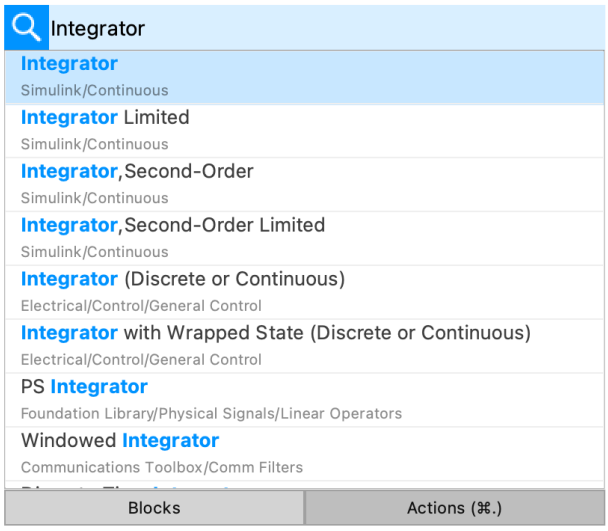


FIGURE 5 – Recherche directe d'un bloc de SIMULINK

3.1 Quelques blocs utiles

On présente ici les blocs qui vous seront utiles durant la session. La librairie SIMULINK est divisée en sous-librairies rassemblant chacune des blocs ayant des types similaires ou des fonctionnalités proches. Il va sans dire que les blocs sont nombreux et que nous n'aborderons ici que les blocs strictement nécessaires. En double-cliquant sur chacun des blocs, une fenêtre de dialogue apparaît permettant de modifier les paramètres du bloc.

La sous-librairie `Simulink/Continuous` (Fig. 6) rassemblent des blocs permettant de simuler les fonctions de transfert (`Transfer Fcn` et `Zero-Pole`), les actions intégrale (`Integrator`) et dérivée (`Derivative`) ainsi que les modèles d'état matriciels (`State-Space`).

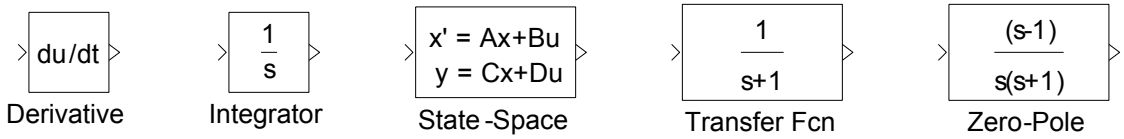


FIGURE 6 – Quelques blocs de la sous-librairie `Continuous`

La sous-librairie `Math Operations` (Fig. 7) contient différents blocs pour traiter les signaux comme multiplier un signal par un gain (`Gain`), multiplier (diviser) et additionner (soustraire) des signaux entre eux (resp. `Product` et `Sum`) ou appliquer une fonction mathématique connue (`Math Function` et `Trigonometric Function`).

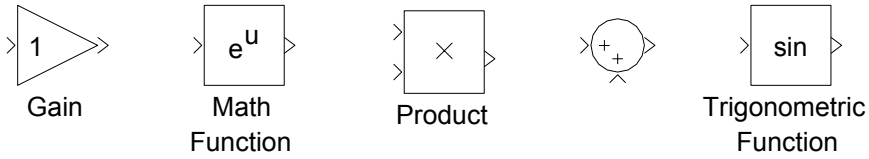


FIGURE 7 – Quelques blocs de la sous-librairie `Math Operations`

On trouve dans la sous-librairie `Signal Routing` (Fig. 8) deux blocs permettant de rassembler des signaux (`Mux`) ou au contraire de séparer les signaux contenus dans un seul fil (`Demux`).

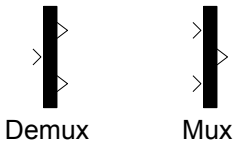


FIGURE 8 – Quelques blocs de la sous-librairie `Signal Routing`

La sous-librairie `Sinks` (Fig. 9) contient les blocs de sorties. En l'occurence, le bloc `Scope` est celui qui nous intéressera particulièrement pour visualiser les signaux mais aussi pour les exporter.

Le bloc `Out` est un bloc particulier qui permet à certaines fonctions de MATLAB de repérer où sont les sorties du schéma.

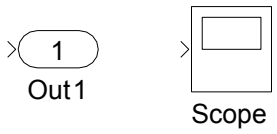


FIGURE 9 – Quelques blocs de la sous-librairie `Sinks`

La sous-librairie `Sources` (Fig. 10) contient les blocs d'entrées. On peut définir différents types de signaux d'entrée comme une constante (`Constant`), une entrée échelon (`Step`), une entrée en rampe (`Ramp`) et une entrée sinusoïdale (`Sine Wave`). Le bloc `In` est un bloc particulier qui permet à certaines fonctions de MATLAB de repérer où sont les entrées du schéma.

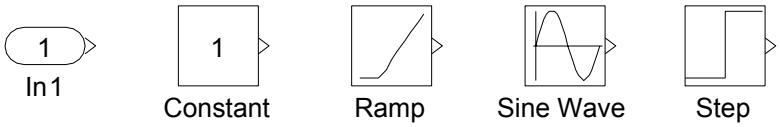


FIGURE 10 – Quelques blocs de la sous-librairie `Sources`

Enfin la sous-librairie `User-Defined Functions` (Fig. 11) contient des blocs polyvalents permettant d'effectuer un traitement du signal plus complexe que les autres blocs. Le bloc `Fcn` permet d'écrire et d'appliquer n'importe quelle fonction rentrée par l'utilisateur, quant à la fonction `Embedded` elle permet d'écrire une fonction complète dans un script, comme on le ferait sous MATLAB.

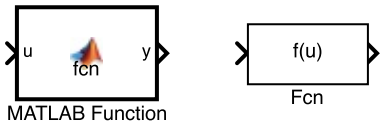


FIGURE 11 – Quelques blocs de la sous-librairie `User-Defined Functions`

3.2 Un premier schéma...

Nous allons à présent créer le schéma SIMULINK de la figure 12.

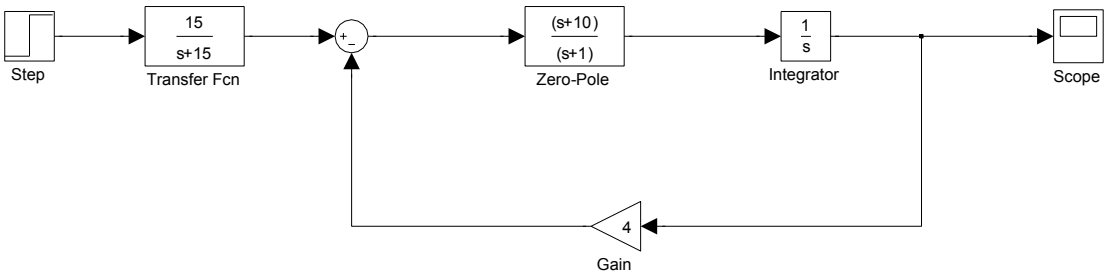


FIGURE 12 – Modèle à reproduire

- Pour créer ce modèle, sélectionnez `New -> Blank Model`.
- Dans la nouvelle fenêtre, utilisez le *Drag-and-Drop* pour y placer les différents éléments constituant votre schéma.
- Ajustez les paramètres de chacun des blocs pour y insérer les bonnes valeurs en double-cliquant dessus.
- Reliez chacun des blocs entre eux.
- Enregistrez votre modèle. Les schémas SIMULINK ont une extension `.mdl` ou `.slx`. N'appelez jamais un programme (script ou fonction) MATLAB dont l'extension est `.m` comme un schéma SIMULINK ; cela créerait un conflit.

3.3 Paramètres de simulation

Avant de procéder à la simulation, nous devons régler les paramètres de simulation comme par exemple le temps de début et fin de simulation, la méthode de résolution ainsi qu'éventuellement la taille du pas de résolution. On accède à la fenêtre de configuration par l'onglet `MODELING`, puis `Model Settings` comme illustré sur la figure 13.

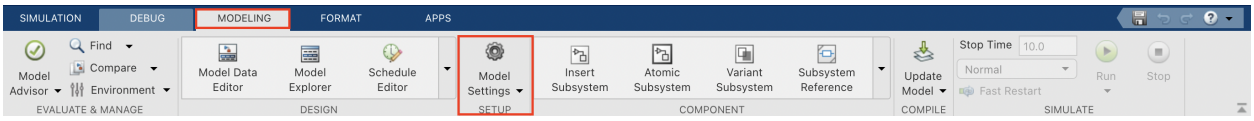


FIGURE 13 – Configuration des paramètres de simulation

La fenêtre (Fig. 14) permet alors de configurer les paramètres de simulation. On laissera le temps de début de simulation (`Start time`) à 0 mais le temps de fin de simulation (`Stop Time`) pourra être ajusté selon le temps de simulation désiré. Par défaut, le type des algorithmes de résolution est réglé sur les méthodes à pas variables (`Variable-Step`) avec les choix d'algorithmes (`Solver`) correspondants.

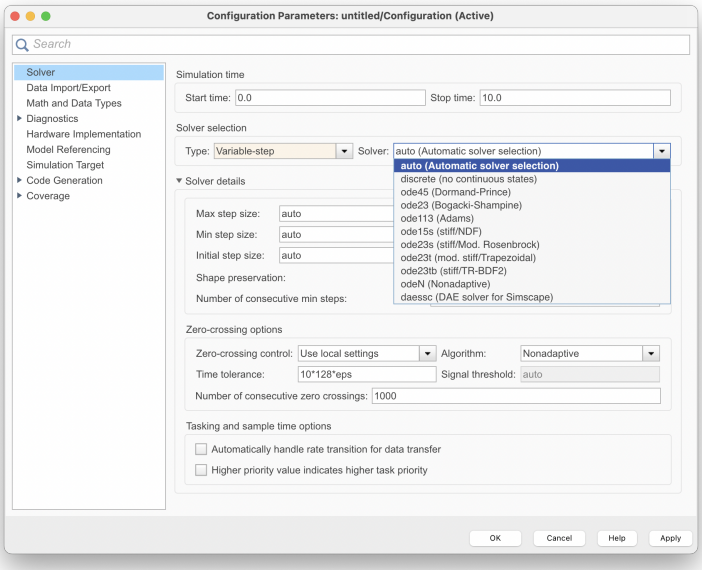


FIGURE 14 – Configuration des paramètres de simulation - Méthodes à pas variable

En modifiant le type à `Fixed-Step`, la fenêtre est actualisée et permet de choisir l'algorithme de son choix (Fig. 15).

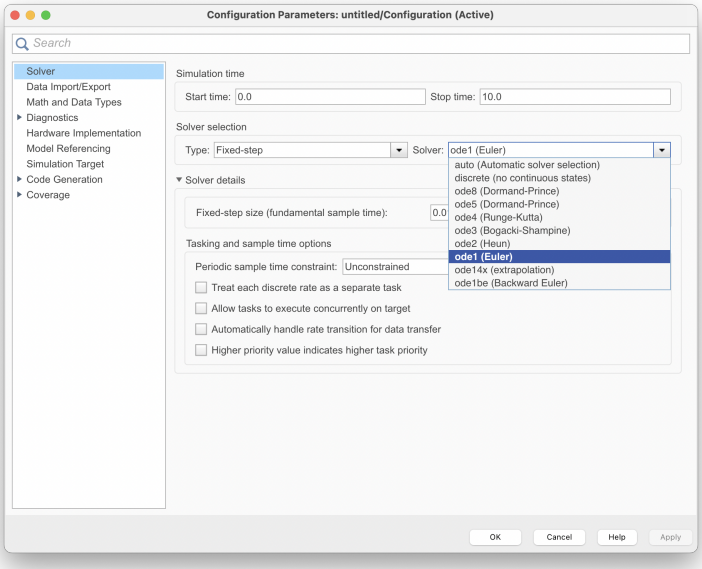


FIGURE 15 – Configuration des paramètres de simulation - Méthodes à pas fixe

On complète à présent les paramètres de simulation du schéma précédent. Les paramètres de simulation sont les suivants :

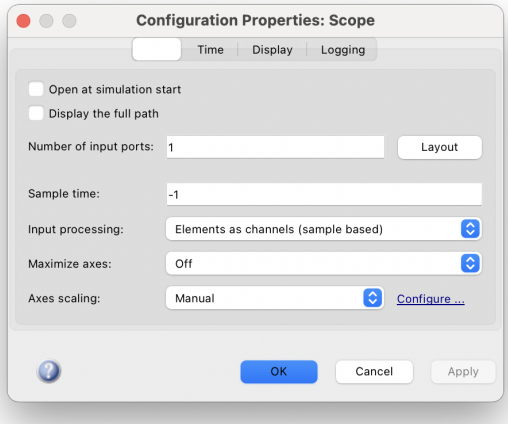
- le temps de simulation est de 15 s
- le type d'algorithme est `Fixed-Step`
- la méthode de résolution est `ode1 (Euler)`
- le pas fixe est de 0.01 s

Le bloc `Step` définit un échelon ayant les caractéristiques suivantes :

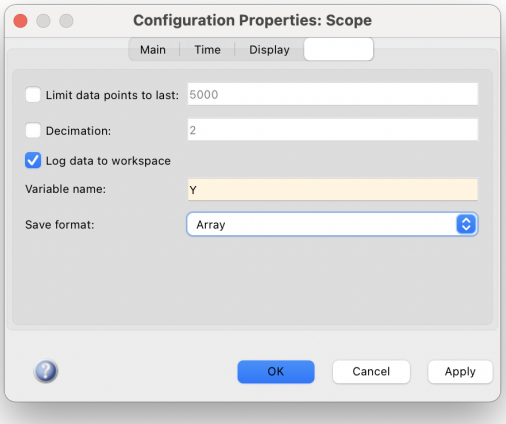
- le front montant (`Step time`) vaut 2 s
- la valeur avant le front montant (`Initial Value`) vaut 1
- la valeur après le front montant (`Final Value`) vaut 4.

3.4 Simulation et exportation des données

Une fois le schéma créé et les paramètres de simulation fixés, on lance la simulation par l'icône `Run`. Le bloc `Scope` permet donc de visualiser la réponse temporelle du système pour l'entrée que nous avons définie. Néanmoins, afin d'exploiter les données calculées, il peut être intéressant d'exporter ces données dans MATLAB sous la forme d'une matrice. La figure 16.a montre la fenêtre de dialogue que l'on obtient en double-cliquant sur l'icône `Parameters` du bloc `Scope`. En cliquant sur le quatrième onglet `Logging`, on obtient la fenêtre de la figure 16.b.



(a) Onglet `Main`



(b) Onglet `Logging`

FIGURE 16 – Fenêtre de dialogue du bloc `Scope`

On y trouve l'option `Limit data points to last` avec une valeur nominale de 5000. Si cette option est cochée, seuls les 5000 derniers points de simulation sont conservés en mémoire et affichés.

Afin de garder les résultats de simulation au complet, on s'assurera que cette option est décochée. Pour exporter les données de simulation, cochez l'option `Save data to workspace`, appelez `Y` la variable d'exportation (`Variable name`) et choisissez par exemple `Array` sur la ligne `Save Format`.

En relançant la simulation, il se crée sous MATLAB la variable structurée `out` qui contient toutes les données de simulation dont celles envoyées au `Scope`. En effet, en tapant `out.Y`, on a accès au contenu de la matrice `Y`. La première colonne de cette matrice `out.Y(:,1)` correspond au vecteur temps et les colonnes suivantes contiennent les valeurs correspondantes des signaux qui ont été envoyés au `Scope`. Lancez la simulation et exécutez la ligne d'instruction suivante sous MATLAB :

```
>> plot(out.Y(:,1),out.Y(:,2))
```

Vous devriez obtenir la même courbe que celle de la fenêtre du `Scope`. Les autres formats exportent les données sous différentes formes de variables structurées.

4 Exercice d'application pour SIMULINK

Nous allons à présent effectuer un exercice pour mettre en pratique nos premières connaissances de SIMULINK. Reproduisez le schéma SIMULINK de la figure 17.

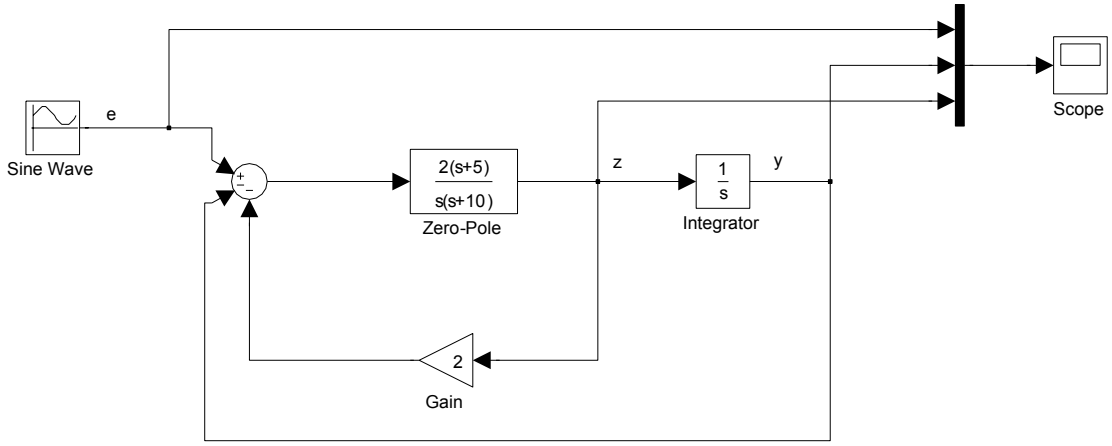


FIGURE 17 – Exercice SIMULINK

Les paramètres de simulation sont les suivants :

- le temps de simulation est de 20 s
- le type d'algorithme est `Fixed-Step`
- la méthode de résolution est `ode4 (Runge-Kutta)`
- le pas fixe est de 0.01 s

Pour l'entrée du système, on désire appliquer le signal sinusoïdal suivant :

$$e(t) = 0.1\cos(5t) + 1$$

Vous devez ajuster les paramètres du bloc `Sine Wave` pour obtenir ce signal car, par défaut, il fournit un sinus. Pour obtenir un cosinus, pensez au déphasage[1]. Lisez attentivement le fonctionnement du bloc pour pouvoir remplir chacune des lignes. Il n'y a rien à modifier sur la ligne `Sample Time`.

Vous devez ensuite exporter les données de simulation et obtenir exactement le graphe de la figure 18. N'oubliez pas d'utiliser la fonction `subplot`. Afin de vous faciliter la tâche, écrivez vos lignes de code dans un script. Cela permet de reexécuter plus rapidement vos commandes en cas d'erreur. Pour ouvrir une fenêtre de script, cliquez sur l'icône de feuille blanche dans la fenêtre MATLAB. La flèche verte de lecture permet d'exécuter le code.

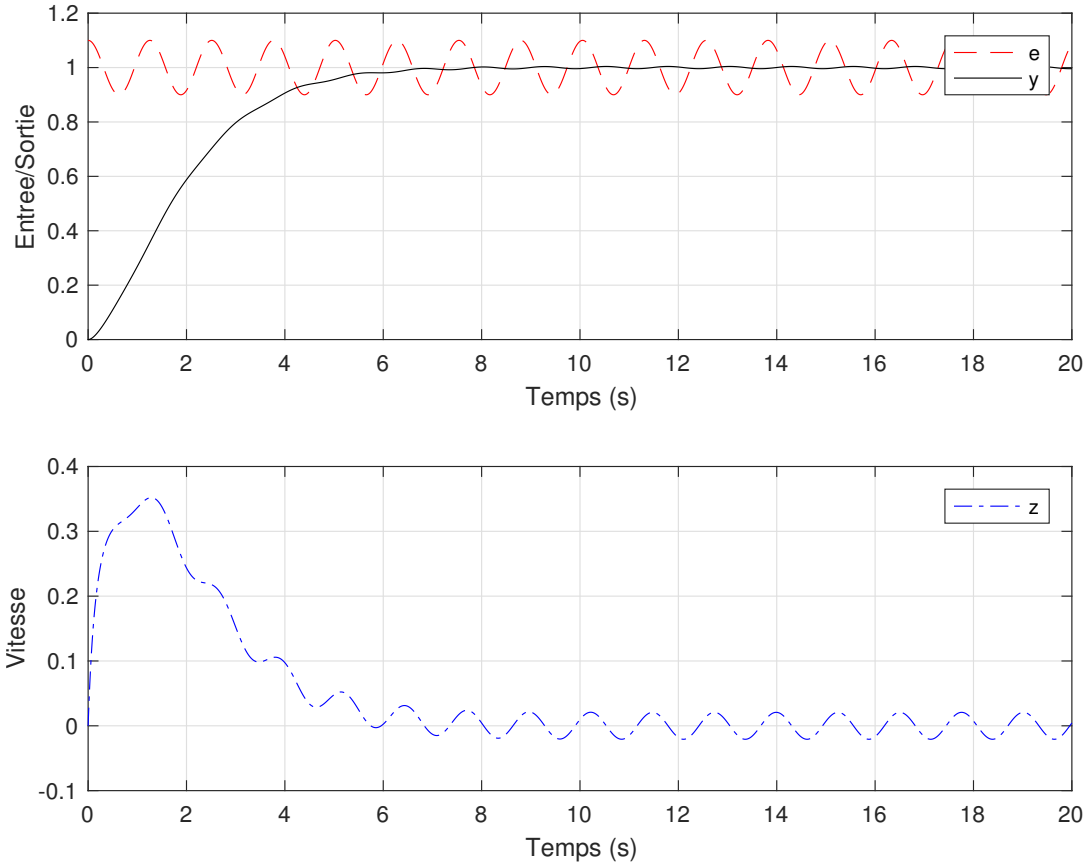


FIGURE 18 – Graphes à obtenir

1. La valeur de π est obtenue avec la commande `pi`. Ne prenez pas de valeur approximative.